

EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPARTMENT OF DATA SCIENCE AND ENGINEERING



EngageMind: AI/ML-Based Language Model Improved on GPT-2, Incorporated in an Interactive, Authenticated App

Supervisor:
Bencsik Gergely
Assistant Professor

Author:
Haseeb Raza
Computer Science BSc

Budapest, 2025

Contents

1	Introduction	4
	Introduction	4
	Background and Motivation	4
	Problem Statement	5
	Aims and Objectives	5
	Scope and Methodology	6
2	User Documentation	7
2.1	Install Guide	7
2.1.1	System Requirements	7
2.1.2	Installation Steps	7
2.2	Configuration and Setup	9
2.2.1	Environment Variables	9
2.2.2	Database Setup	10
2.2.3	Starting Services	10
2.3	User Interaction with the EngageMind RAG Application	11
2.3.1	Introduction to the Frontend Interface	11
2.3.2	User Registration	11
2.3.3	Logging In	14
2.3.4	Chatting with the RAG System	15
2.3.5	Fine-Tuning the Model on Your Documents	19
2.3.6	Troubleshooting	20
2.3.7	Example Commands	22
3	Developer Documentation	24
3.1	Cloning the Repository	24
3.1.1	Prerequisites for Cloning	24
3.1.2	Repository Details	25
3.1.3	Cloning the Repository	25
3.1.4	Verifying the Local Repository	25

3.1.5	Repository Structure Overview	25
3.1.6	Handling Repository Updates	25
3.1.7	Managing Submodules (If Applicable)	25
3.1.8	Troubleshooting Common Issues	26
3.2	Prerequisites	26
3.2.1	MongoDB (Database)	26
3.2.2	Flask Backend (RAG + Inference Engine)	27
3.2.3	Express Backend (User Authentication)	28
3.2.4	React Frontend	28
3.2.5	Background Services: Celery and Redis	29
3.3	External APIs Used	30
3.3.1	Mistral API	30
3.3.2	Tavily Web Search API	31
3.3.3	Resend API (Email Delivery)	31
3.3.4	Google OAuth (Passport.js)	32
3.4	Software Architecture	33
3.4.1	System Overview	33
3.4.2	Architecture Diagram	34
3.4.3	System Logic Overview	34
3.4.4	RAG System Details	35
3.4.5	LangGraph Memory Workflow (<code>app.py</code>)	36
3.4.6	Authentication System	36
3.4.7	Database Design (MongoDB)	37
3.4.8	API Surface	37
3.4.9	Status Feedback & Evaluation	37
3.4.10	Deployment Considerations	38
3.5	RAG Implementation	38
3.6	UserAuth Backend Implementation	39
3.6.1	User Authentication Backend Summary	41
3.7	Fine-Tune API Implementation	43
3.7.1	Service Overview	43
3.7.2	API Endpoints	43
3.7.3	Training Workflow	44
3.7.4	Frontend Integration	44
3.8	Frontend Implementation	45
3.8.1	Project Overview	45
3.8.2	Application Bootstrapping: <code>index.js</code>	45

3.8.3	Routing Architecture: <code>App.js</code>	45
3.8.4	User Authentication Flow	45
3.8.5	ProtectedRoute Logic	48
3.8.6	Chat System Detailed Flow	48
3.8.7	Components Connection	51
3.8.8	Loading and Error Handling	51
3.8.9	LocalStorage Key List	52
3.8.10	Data Flow Diagram	52
3.8.11	Chat Application Flow	52
3.9	Concrete Improvements	52
3.9.1	Harden GPT-2 LoRA Live Chat	52
3.9.2	Persist Training State Across Page Refresh	54
3.9.3	Keep Expanding the Production Test Layer	54
3.9.4	Make Document Ingestion More Honest About PDFs	55
3.9.5	Add a Startup Configuration Check	55
3.9.6	Keep User Data Boundaries Explicit	56
3.9.7	Improve Model Lifecycle Management	56
3.9.8	Tighten the Demo Story	56
	Acknowledgements	58
	Bibliography	58

Chapter 1

Introduction

Introduction

Conversational AI systems are now good at producing fluent text, but fluency alone is not enough for a useful assistant. Models such as GPT-2 [1] can generate coherent responses, yet they do not remember a user’s earlier sessions unless another system stores that context for them. They also cannot use a user’s private documents by default. As a result, a conversation may sound natural for a few turns but still feel generic, repetitive, or disconnected from the user’s actual material.

This thesis presents **EngageMind**, a working web application that adds these missing pieces around a language-model workflow. It includes user authentication, persistent conversations, document upload, retrieval-augmented generation (RAG), and a GPT-2 LoRA fine-tuning path with visible task status plus a GPT-2 LoRA chat mode that can use the saved adapter after training. The goal is not to claim that one model solves personalization on its own. The goal is to show how a practical full-stack system can combine storage, retrieval, authentication, and training feedback so that a user can return to previous conversations and ask questions about their own documents.

Background and Motivation

Large language models such as GPT-2, GPT-3, and GPT-4 have changed what users expect from chat interfaces. They can answer questions, summarize text, and produce convincing explanations. However, most model APIs are stateless at the point of use: each request must be given the context it needs. If the application does not store conversation history, uploaded files, or user identity, the model cannot reliably continue a previous session.

This matters in ordinary use cases. A student may want to upload lecture notes and ask follow-up questions over several sessions. A developer may want the assistant to remember earlier design decisions. A user may also need clear feedback when a long-running training job is queued, running, or failed. These are application-level requirements, not just model-level requirements.

EngageMind was built around this gap. It keeps user accounts separate, stores chat history in MongoDB, indexes uploaded documents with FAISS, and exposes a fine-tuning workflow through a small status panel. In this way, the project studies the engineering around personalization: how data enters the system, how it is retrieved, how memory is preserved, and how the user sees progress.

Problem Statement

The problem addressed in this thesis is the gap between a capable text generator and a usable personalized assistant. A model can answer a single prompt, but an application must decide who the user is, what documents belong to that user, which conversation is active, and which previous messages should be available. Without these pieces, users repeat context and receive answers that are not grounded in their own material.

There is also a deployment problem. Uploaded documents and conversation history are personal data, so they must be tied to authentication and kept separate between users. Training is another concern: it is slower than a normal chat request, so the interface must show whether the job is waiting, running, completed, or failed. A silent background process is not enough for a user-facing system.

EngageMind addresses these issues as an integrated prototype. It combines an Express authentication service, a React frontend, Flask APIs for RAG and fine-tuning, MongoDB for persistence, FAISS for retrieval, and Redis/Celery for background training jobs.

Aims and Objectives

The primary goal of this thesis is to design, implement, and evaluate **EngageMind** as a personalized conversational application. The work focuses on memory, document grounding, authentication, and visible training status. The main objectives are:

- **Authentication and Sessions:** Implement registration, login, email verification, password reset, Google OAuth, and JWT-protected routes.
- **Conversation Memory:** Store conversations so users can return to previous chats instead of starting from zero each time.
- **Document-Grounded Answers:** Let users upload text, Markdown, PDF, and DOCX files, then retrieve relevant chunks during chat.
- **Training Status:** Expose GPT-2 LoRA fine-tuning as a background task with visible states such as pending, progress, success, and failure.
- **Full-Stack Integration:** Connect the React frontend, Express auth backend, Flask RAG API, Flask fine-tune API, MongoDB, FAISS, Redis, Celery, PyTorch, and PEFT

in one working prototype.

- **Evaluation and Verification:** Validate the main flows through implementation checks, reproducible tests, and documented demo scenarios.

Scope and Methodology

This research focuses on EngageMind as a functional proof of concept. The implemented system covers authentication, chat history, document upload, RAG responses, and a GPT-2 LoRA training workflow. The prototype is intended for text-based interaction in English. Large-scale benchmarking, production deployment, and broad multilingual evaluation are outside the scope of this thesis.

The implementation is modular. React handles the browser interface. Express handles accounts and JWTs. Flask handles document ingestion, RAG chat, and fine-tune task submission. MongoDB stores users, documents, conversations, and task metadata. FAISS stores per-user vector indexes, while PyTorch and PEFT are used for GPT-2 LoRA training.

The expected outcome is a prototype where an authenticated user can return to earlier conversations, upload documents, ask grounded questions, and start a fine-tuning job with visible progress. The value of the system comes from combining these pieces into one reproducible flow.

This chapter introduced the motivation for EngageMind and the problem it addresses. The next chapters describe how the system is installed, how users interact with it, and how the implementation is organized.

Chapter 2

User Documentation

2.1 Install Guide

This chapter explains how to install and use EngageMind. It is written for a local thesis/demo setup where the frontend, authentication backend, RAG API, fine-tune API, MongoDB, and Redis run on the same machine.

2.1.1 System Requirements

Before installation, the machine should meet these basic requirements:

Hardware Requirements

- **Processor:** Minimum 2.0 GHz dual-core; quad-core recommended
- **RAM:** At least 8 GB required; 16 GB or more recommended
- **Storage:** Minimum 20 GB of free disk space; SSD recommended
- **GPU (Optional):** A CUDA-compatible GPU can speed up training, but the demo can run without one

Software Requirements

- **Operating System:** Linux (Ubuntu 20.04+), macOS (10.15+), or Windows 10/11
- **Python:** Version 3.10 or higher recommended
- **Node.js:** Version 18 or higher recommended
- **Git:** For repository management
- **MongoDB:** Required for users, documents, chats, and task metadata
- **Redis:** Required for Celery-based fine-tuning tasks

2.1.2 Installation Steps

Installing Git

Install Git based on your operating system:

Windows:

1. Download the Git installer from <https://git-scm.com/download/win>
2. Run the installer with default settings
3. Verify with: `git -version`

macOS:

1. Open Terminal and type `git -version`
2. If not installed, macOS will prompt to install Xcode Command Line Tools

Linux:

```
1 sudo apt-get update
2 sudo apt-get install git -y
3 git --version
```

Cloning the Repository

Clone the EngageMind repository:

```
1 git clone https://github.com/haseebraza715/EngageMind.git
2 cd EngageMind
```

Setting Up Python and Flask

Set up the Flask RAG and fine-tune services from the repository root:

```
1 # Verify Python installation
2 python3 --version
3
4 # Create and activate virtual environment
5 cd engagemind-rag
6 python3 -m venv .venv
7 # For macOS/Linux:
8 source .venv/bin/activate
9 # For Windows:
10 .venv\Scripts\activate
11
12 # Install dependencies
13 pip install --upgrade pip
14 pip install -r requirements.txt
```

Setting Up Node.js and Express

Set up the Express authentication service from the repository root:

```
1 # Verify Node.js installation
2 node --version
3 npm --version
4
5 # Install Express dependencies
6 cd engagemind-backend
7 npm install
```

Setting Up React Frontend

Set up the React frontend from the repository root:

```
1 # Install React dependencies
2 cd engagemind-frontend
3 npm install
```

2.2 Configuration and Setup

After installing dependencies, create the environment files and start the required services.

2.2.1 Environment Variables

Create environment files for the backend and RAG services. The two `JWT_SECRET` values must match because the backend issues the token and the RAG/fine-tune services verify it.

Flask RAG/Fine-Tune Configuration (engagemind-rag/.env):

```
1 MONGO_URL=mongodb://localhost:27017/demo_db
2 JWT_SECRET=your_jwt_secret
3 MISTRAL_API_KEY=your_mistral_api_key
4 CELERY_REDIS_URL=redis://localhost:6379/0
```

Express Auth Configuration (engagemind-backend/.env):

```
1 PORT=5003
2 SESSION_SECRET=your_express_session_secret
3 JWT_SECRET=your_jwt_secret
```

```
4 MONGO_URI=mongodb://localhost:27017/engagemindbackend
```

2.2.2 Database Setup

MongoDB stores authentication data, uploaded documents, conversations, and fine-tune metadata. A local setup can use MongoDB on `localhost:27017`.

```
1 # Install MongoDB
2 sudo apt-get install -y mongodb-org
3
4 # Start MongoDB service
5 sudo systemctl start mongod
6 sudo systemctl enable mongod
7
8 # Create database and user
9 mongosh
10 use engagemindbackend
11 db.createUser({
12     user: "engagemind_user",
13     pwd: "engagemind_pass",
14     roles: [{ role: "readWrite", db: "engagemindbackend" }]
15 })
16
17 # Update connection strings in engagemind-backend/.env and
18     engagemind-rag/.env
18 MONGO_URI=mongodb://localhost:27017/engagemindbackend
19 MONGO_URL=mongodb://localhost:27017/demo_db
```

2.2.3 Starting Services

The easiest way to start the project is from the EngageMind root:

```
1 ./scripts/run_all.sh
```

For manual startup, run the services below in separate terminals:

RAG API:

```
1 cd engagemind-rag
2 source .venv/bin/activate # macOS/Linux
3 python main.py
```

Express Authentication:

```
1 cd engagemind-backend
2 npm start
```

Fine-Tune API and Worker:

```
1 cd engagemind-rag
2 source .venv/bin/activate # macOS/Linux
3 python fine_tune/fine_tune_app.py
4 celery -A fine_tune.celery_config.app worker --pool=solo --
    concurrency=1 --loglevel=info
```

React Frontend:

```
1 cd engagemind-frontend
2 npm start
```

Access the application at <http://localhost:3000>

2.3 User Interaction with the EngageMind RAG Application

2.3.1 Introduction to the Frontend Interface

The frontend is the part of EngageMind that the user sees. From the browser, a user can create an account, log in, open the chat page, upload documents, ask questions, and start a fine-tuning job. The main user path is:

- **User Registration:** Creating an account and completing the available email verification flow.
- **Logging In:** Storing a JWT token in the browser for protected routes.
- **Uploading Documents:** Adding source material for RAG answers.
- **Chatting with the System:** Asking questions inside a saved conversation.
- **Fine-Tuning:** Starting a GPT-2 LoRA job, checking its status, and switching to GPT-2 LoRA chat mode after completion.

The following sections describe those steps from the user's point of view.

2.3.2 User Registration

Users need an account before they can open the chat page. Registration collects a username, email address, and password, then creates a local user account in the Express backend.

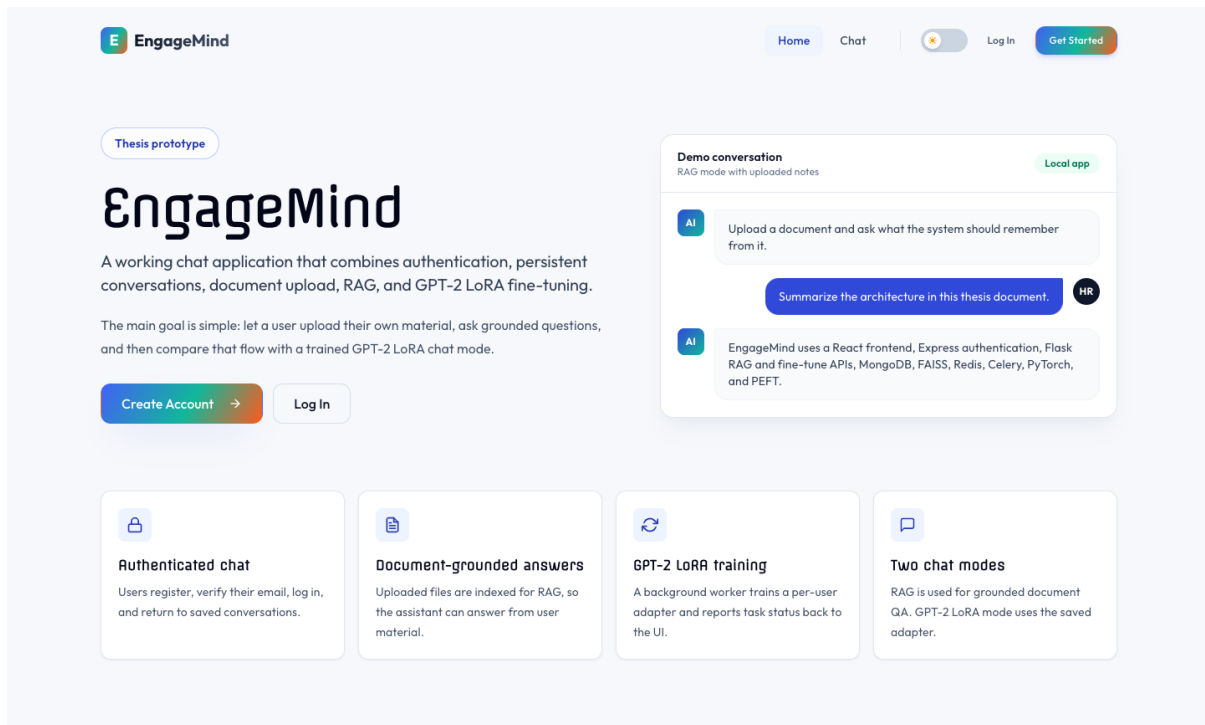


Figure 2.1: The simplified EngageMind landing page, showing the thesis prototype purpose and direct links for account creation and login.

Step-by-Step Registration Process

1. **Open the Registration Page:** Use the Register button on the landing page, or open <http://localhost:3000/register>.
2. **Complete the Registration Form:**
 - **Input Fields:** Users must provide a username, a valid email address, and a password with at least six characters.
 - **Example Form:**
Username: johndoe
Email: user@example.com
Password: *****
 - **Validation:** The backend rejects missing fields, invalid email format, duplicate usernames/emails, and passwords shorter than six characters.
3. **Submit the Form:** Click the registration button to create the account.
4. **Verify Email Address:**

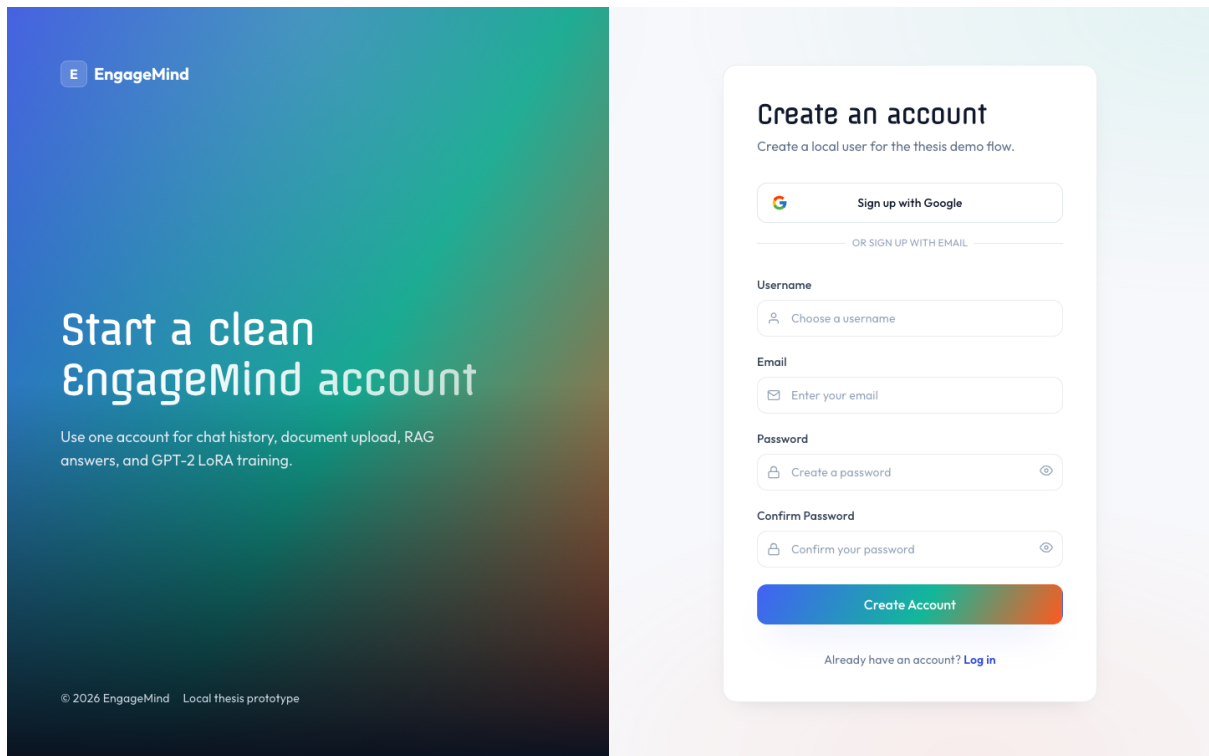


Figure 2.2: Register Page

- Upon successful submission, the backend creates a verification token and attempts to send a verification email if email delivery is configured.
- Users can open the email and click the link to mark the account as verified.
- In the local thesis demo flow, the backend also returns the verification token. The frontend uses it to open the verification page without depending on real email delivery.

Handling Registration Errors

The most common registration problems are simple input errors:

- **Duplicate Email:** If the error “Email already taken” appears, users should try a different email or use the “Forgot Password” option to recover an existing account.
- **Invalid Input:** Errors like “Invalid email format” or “Password must be at least 6 characters long” prompt users to correct their entries based on the provided guidelines.

After registration and verification, the user can log in and open the protected chat route.

2.3.3 Logging In

Users can log in with an email/username and password. Google login is also available when OAuth credentials are configured in the backend environment.

Step-by-Step Login Process

1. **Open the Login Page:** Use the Login button on the landing page, or open `http://localhost:3000/login`.

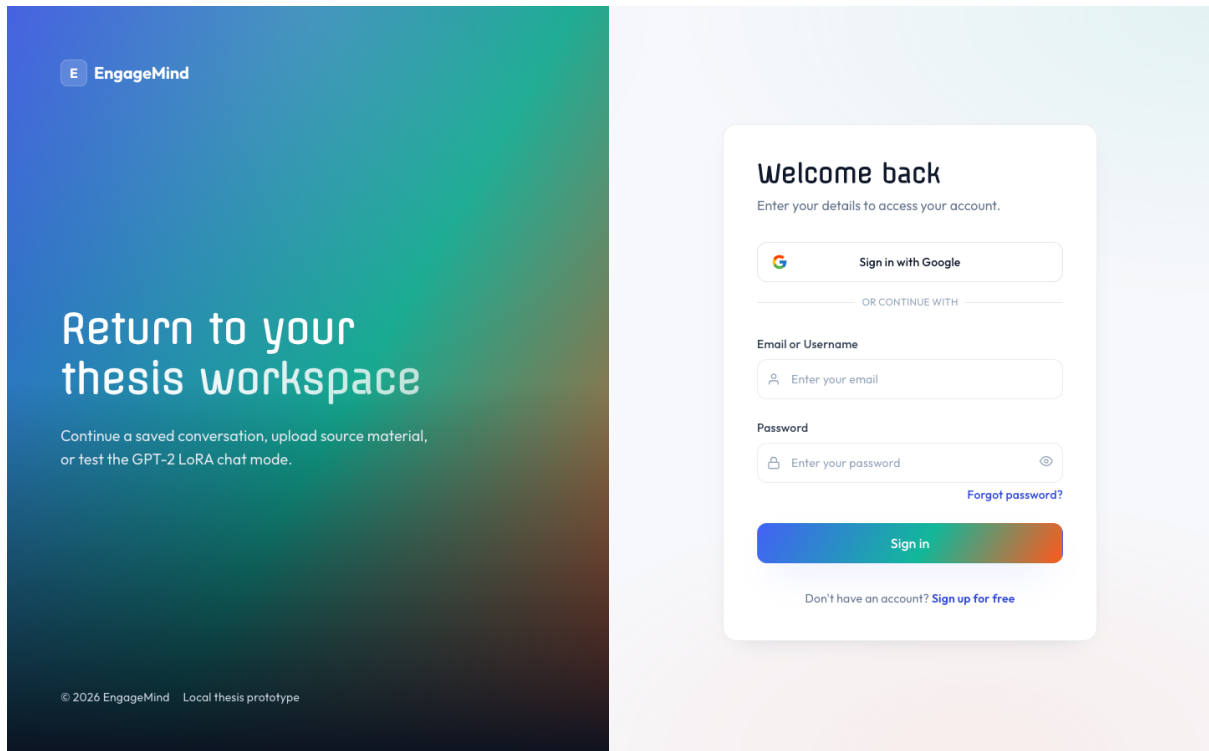


Figure 2.3: The login page of the EngageMind RAG application, featuring email/password login, Google sign-in, and password recovery options.

2. Choose a Login Method:

- **Standard Login:** Enter the registered email address or username and password.
- **Google Login:** Click the “Sign in with Google” button to authenticate via Google. This requires Google OAuth credentials to be configured in the Express backend. Upon success, the user is redirected to the frontend OAuth success route.

3. Enter Credentials (Standard Login):

- **Input Fields:**

Email or Username: `user@example.com`

Password: *****

- **Submit:** Click the “Login” button to authenticate.

4. Troubleshoot Invalid Logins:

- **Error Handling:** If an error such as “Invalid credentials” appears, users should check for typos or incorrect casing in their input.
- **Case Sensitivity:** Passwords are case-sensitive; ensure that Caps Lock is disabled.

5. Password Recovery:

- A “Forgot Password?” link is provided below the login form.
- Clicking this link takes users to the password recovery page, where they can enter their registered email address.
- An email will be sent to the provided address with a secure link to reset the password.
- After setting a new password, users can return to the login page and authenticate with their updated credentials.

After a successful login, the frontend stores the JWT token in `localStorage` and redirects the user to the chat page.

2.3.4 Chatting with the RAG System

The chat page is the main workspace. A logged-in user can create or select a conversation, upload source documents, ask questions, and return later to the saved message history. If the user has not uploaded a document yet, the assistant gives upload guidance instead of pretending to know the user’s material.

Accessing the Chat Interface

Only authenticated users can use the chat feature. If a user opens `/chat` without a token, the frontend shows an access-denied card or redirects to login depending on the route.

Interacting with the Chat Interface

Once the chat page is open, the usual flow is:

1. Upload Documents (Optional):

- **Open Upload:** Click the upload control in the chat interface.

- **Select and Upload:** Choose a file from your device (.txt, .md, .pdf, or .docx, maximum 10 MB) and click “Upload.”
- **System Feedback:** A successful upload is stored in MongoDB and added to the user’s FAISS index.
- **Error Handling:** Files over 10 MB are rejected. Image-only PDFs may not produce usable text because the current parser extracts text rather than running OCR.

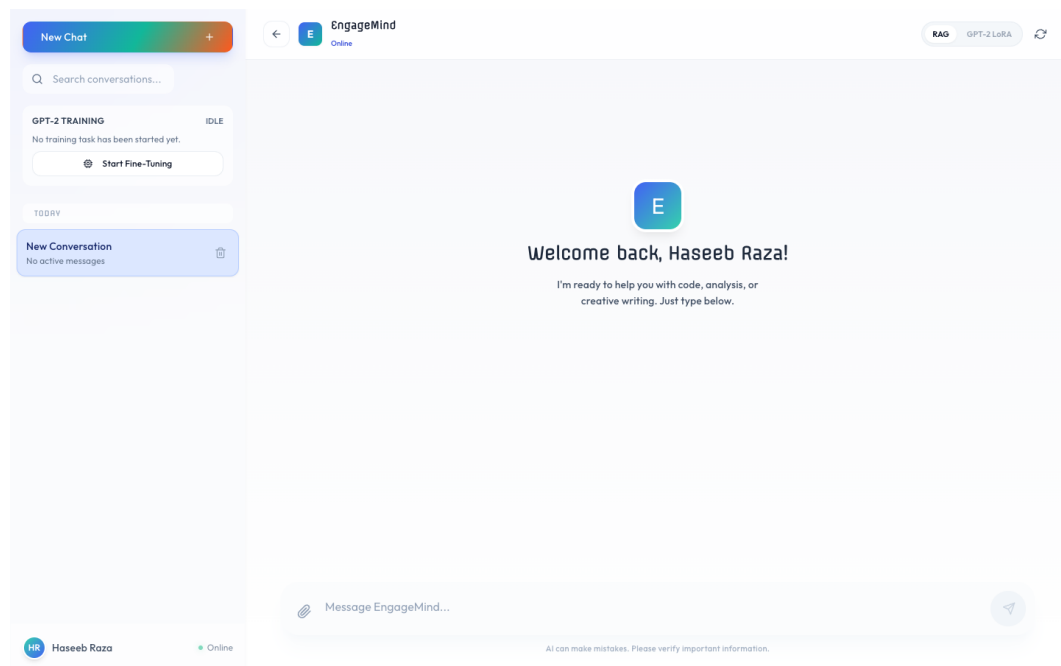


Figure 2.4: The EngageMind chat interface with conversation history, input field, and upload icon.

2. Enter a Query:

- **Text Input:** Type a question into the chat input field. For example:
`Can you summarize the key ideas in the uploaded document?`
- **Context Awareness:**
 - If documents are uploaded, the RAG API retrieves relevant chunks and generates an answer.
 - If no documents are uploaded, the assistant asks for source material first.

3. Review Responses:

- **Example (Document Summary):**

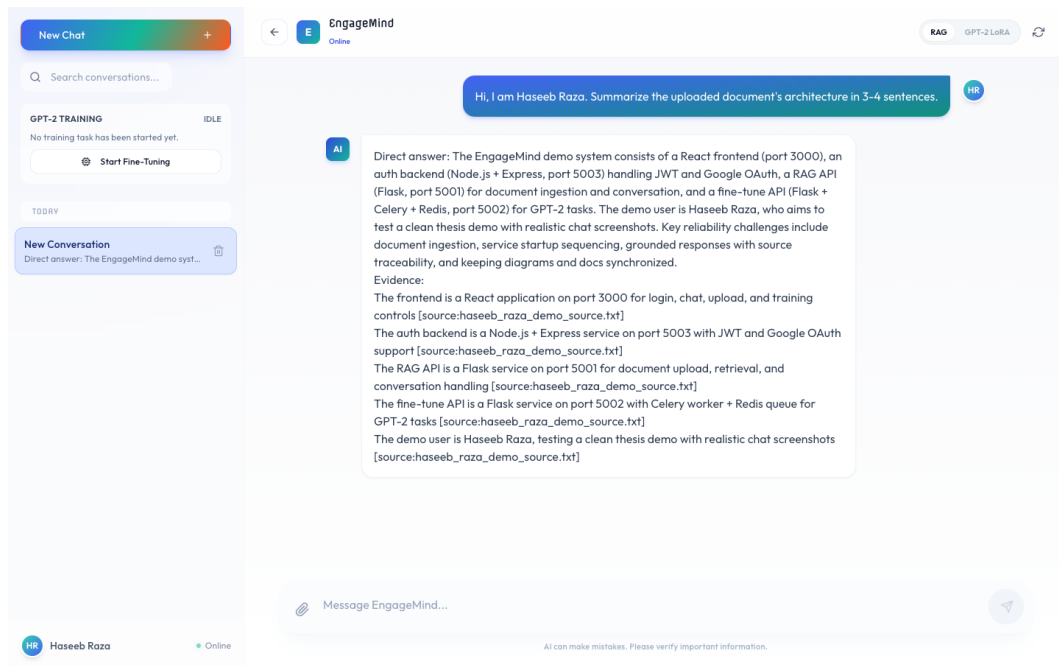


Figure 2.5: Example RAG conversation using the demo user Haseeb Raza and an uploaded thesis source document.

- **User:** “Summarize the uploaded document’s architecture and component responsibilities in 3–4 sentences.”
- **System:** “EngageMind uses a React frontend, an Express auth backend, a Flask RAG API, and a Flask fine-tune API with Celery and Redis. The answer summarizes the services described in the uploaded source.”
- **Example (Challenge Extraction):**
 - **User:** “List the main reliability and thesis-alignment challenges from the document.”
 - **System:** “The assistant lists concrete issues from the uploaded source, such as index reliability, service startup, auth consistency, and keeping diagrams aligned with the code.”
- **Example (Fact Lookup in Uploaded Document):**
 - **User:** “According to the uploaded document, who introduced the transformer model and in which year?”
 - **System:** “The assistant returns: Vaswani et al., 2017, based on the uploaded source text.”

4. Refine and Follow Up:

- Ask follow-up questions in the same conversation.

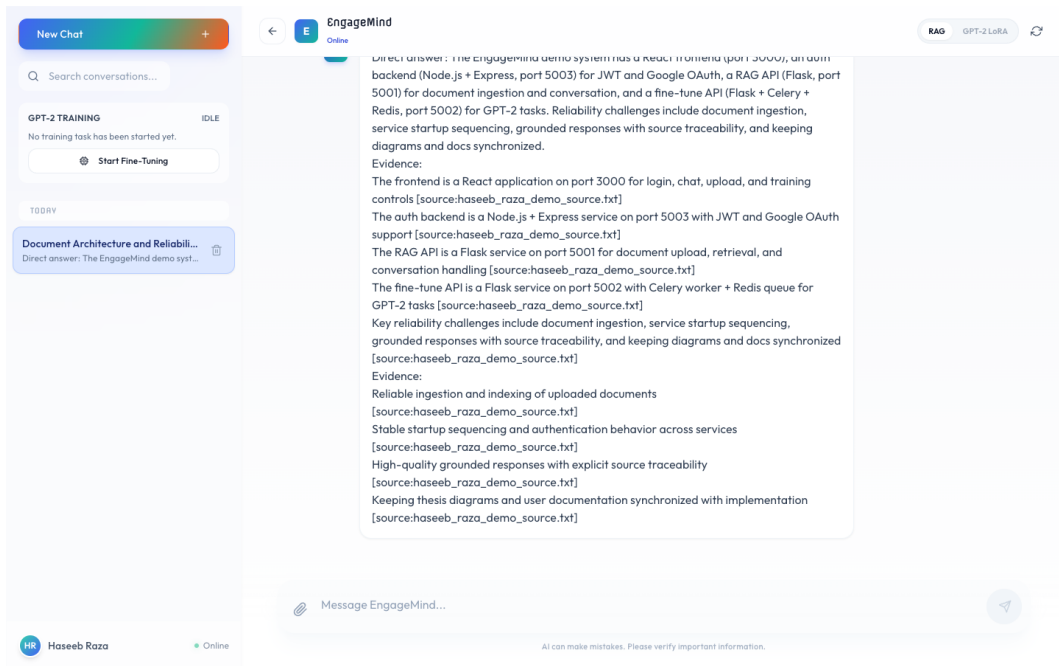


Figure 2.6: Assistant response extracting key challenges from the uploaded source document.

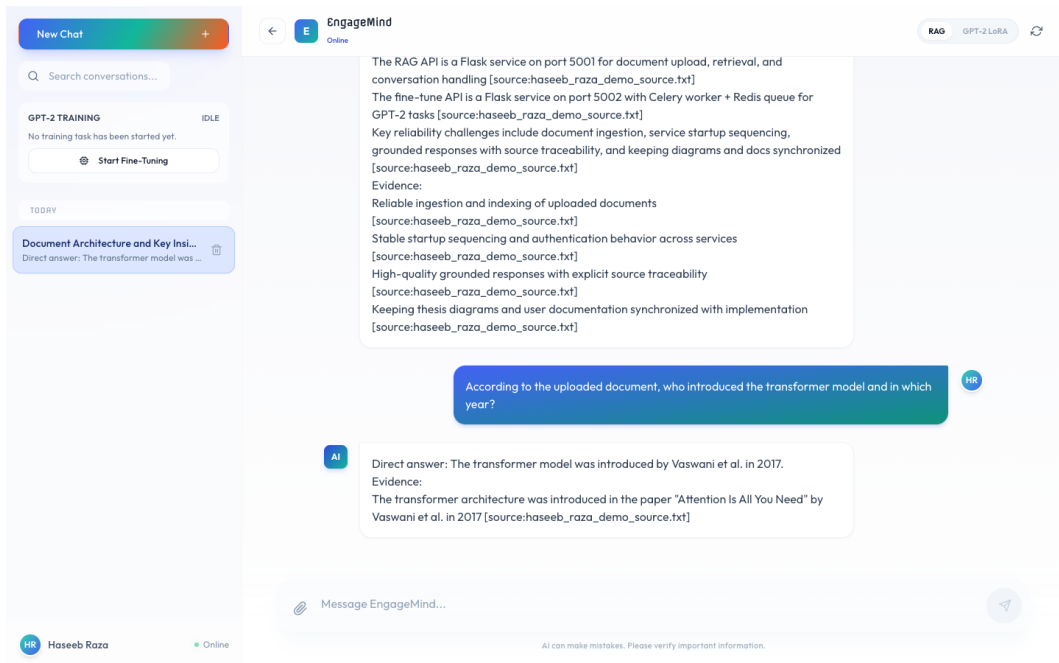


Figure 2.7: Assistant response to a fact lookup question grounded in the uploaded source document.

- Refresh the page and reopen the conversation to confirm that history is still available.

The important behavior is that chat is tied to the authenticated user. Messages are stored in MongoDB, document vectors are kept in a per-user FAISS index, and the frontend uses the same JWT token for chat requests.

2.3.5 Fine-Tuning the Model on Your Documents

EngageMind also exposes a GPT-2 LoRA training path. The user starts the job from the chat sidebar, and the fine-tune API queues the work through Celery. After training succeeds, the chat interface can switch to a dedicated GPT-2 LoRA mode while RAG remains the default for grounded document QA.

Prerequisites

- At least one document must be uploaded before starting training (see Section 2.3.4).
- The Fine-Tune API service and the Celery worker must be running. Start them with:

```
1 cd engagemind-rag
2 source .venv/bin/activate # macOS/Linux
3 python fine_tune/fine_tune_app.py
4 celery -A fine_tune.celery_config.app worker --pool=solo --
    concurrency=1 --loglevel=info
```

- Redis must be running on localhost:6379 (required by Celery).

Step-by-Step Fine-Tuning Process

1. **Open Chat:** Log in and go to <http://localhost:3000/chat>.
2. **Upload Documents:** Upload at least one readable document. Empty corpora are rejected.
3. **Start Training:** Click “**Start Fine-Tuning**” in the sidebar. The frontend calls `/api/fine-tune`.
4. **Monitor Progress:** The sidebar displays live status updates:
 - **PENDING** — Task is queued and waiting to start.
 - **PROGRESS** — Training is running (tokenisation, LoRA setup, gradient updates).
 - **SUCCESS** — Training completed; adapter weights saved.
 - **FAILURE** — An error occurred; check the Celery worker logs.

5. **Training Complete:** Once the status shows **SUCCESS**, the adapter and tokenizer are saved under `models/{user_id}/gpt2-lora/`.
6. **Switch Modes:** Use the chat mode selector in the header to switch from **RAG** to **GPT-2 LoRA**. The GPT-2 LoRA option is disabled if no adapter is available.
7. **Send a Message:** Ask a question in GPT-2 LoRA mode to generate a response from the trained adapter.

What Happens During Training

- All text from uploaded documents is extracted and normalised.
- Texts are tokenised with a maximum sequence length of 256 tokens.
- A LoRA adapter is applied to GPT-2 with rank `r=16` and `lora_alpha=32`.
- Training runs for 1 epoch with batch size 1 (suitable for CPU or low-memory GPU).
- The adapter weights and tokeniser are stored for reuse.

Troubleshooting Fine-Tuning

- **Status stuck at PENDING:** Ensure the Celery worker is running and Redis is active.
- **FAILURE status:** Check the Celery worker terminal output for error details. Common causes are insufficient memory or empty document corpus.
- **No documents available:** Upload at least one document before starting training.

2.3.6 Troubleshooting

Most local problems come from missing services, wrong environment values, or ports already being used. The checks below cover the cases most likely to appear during a demo.

Common Issues and Solutions

- **Port Already in Use:**
 - **Symptom:** The server fails to start because the port is already in use.
 - **Solution:**
 - * Prefer stopping the old process before changing ports, because the frontend clients expect ports 5001, 5002, and 5003 by default.
- **Linux/macOS:**

```
1 | lsof -i :3000
```

```
2 kill -9 <PID>
```

- **Windows:**

```
1 netstat -ano | findstr :3000
2 taskkill /PID <PID> /F
```

- **Missing Dependencies:**

- **Symptom:** Errors indicating missing packages during startup.

- **Solution:**

- * For Node.js components, reinstall dependencies:

```
1 npm install
```

- * For Python dependencies in the RAG component:

```
1 pip install -r requirements.txt
```

- **Authentication or CORS Issues:**

- **Symptom:** The frontend logs the user out or chat requests return 401.

- **Solution:** Check that `JWT_SECRET` is the same in `engagemind-backend/.env` and `engagemind-rag/.env`.

- **Frontend Does Not Load:**

- **Symptom:** The browser displays a blank page or fails to render the application.

- **Solution:**

- * Check the console log for build errors.

- * Clear the browser cache.

- * Ensure the frontend has been started with `npm start`.

- * Ensure the backend, RAG API, and fine-tune API are reachable on their documented ports.

OS-Specific Notes

- **Windows:**

- Run terminals as Administrator if required.

- Ensure firewall or antivirus software does not block Node.js or Python processes.

- **Linux/macOS:**

- Use `sudo` for installation commands if permission issues arise.
- Adjust file permissions if errors occur.

These checks are enough for the local thesis run. For a full verification pass, use the repository script:

```
1 ./scripts/verify_all.sh
```

2.3.7 Example Commands

The following commands are useful when starting or checking only the RAG API.

Example Terminal Commands

On Linux or macOS:

- **Linux/macOS RAG Setup:**

```
1 # From the repository root
2 cd engagemind-rag
3
4 # Create and activate a Python virtual environment
5 python3 -m venv .venv
6 source .venv/bin/activate
7
8 # Install dependencies from requirements.txt
9 pip install -r requirements.txt
10
11 # Start the RAG component
12 python main.py
```

- **Windows RAG Setup:**

```
1 REM Navigate to the 'rag' folder
2 cd EngageMind\engagemind-rag
3
4 REM Create and activate a Python virtual environment
5 python -m venv .venv
6 .venv\Scripts\activate
7
8 REM Install dependencies
9 pip install -r requirements.txt
10
11 REM Start the RAG component
12 python main.py
```

- **API Health Check:** To verify the RAG backend server is operational, users can run:

```
1 curl http://localhost:5001/api/health
```

This command returns a JSON response with service status and dependency checks. A healthy response uses "status": "healthy"; if MongoDB is unavailable, the endpoint returns an unhealthy status with HTTP 503.

These commands provide a practical reference for setting up and verifying the application's components, complementing the visual aids provided throughout this guide.

Chapter 3

Developer Documentation

3.1 Cloning the Repository

The source code is stored in the **EngageMind** repository. A developer needs the full repository because the frontend, backend, RAG service, fine-tune service, scripts, diagrams, and documentation are kept together.

3.1.1 Prerequisites for Cloning

Before cloning, ensure that Git is installed on your development machine:

- **Windows:**

- The EngageMind repository is hosted on GitHub [2], and requires Git [3] to be installed.
- Open a Command Prompt or Git Bash to verify installation by running:

```
1 git --version
```

- **macOS:**

- macOS users may opt to use Homebrew [4] to install Git or other dependencies.
- Verify by running in the Terminal:

```
1 git --version
```

- **Linux (Ubuntu/Debian):**

- Update your package lists and install Git:

```
1 sudo apt-get update
2 sudo apt-get install git -y
```

- Confirm the installation with:

```
1 git --version
```

3.1.2 Repository Details

The repository URL is:

```
1 https://github.com/haseebraza715/EngageMind.git
```

The thesis work uses the `main` branch unless a separate review branch is created.

3.1.3 Cloning the Repository

Clone the repository and enter the project folder:

```
1 git clone https://github.com/haseebraza715/EngageMind.git
2 cd EngageMind
```

3.1.4 Verifying the Local Repository

After cloning, check the working tree and remote:

```
1 git status
2 git remote -v
```

3.1.5 Repository Structure Overview

The main folders are:

- `/engagemind-frontend`: React app with pages, components, and API clients.
- `/engagemind-backend`: Express service for authentication, profile, and admin routes.
- `/engagemind-rag`: Flask RAG API, fine-tune API, Celery tasks, and Python tests.

3.1.6 Handling Repository Updates

To update your local repository:

```
1 git pull origin main
```

Fetches and merges the latest commits from the `main` branch.

3.1.7 Managing Submodules (If Applicable)

If the repository uses submodules:

1. Initialize Submodules:

```
1 git submodule update --init --recursive
```

Ensures submodules are downloaded and initialized.

3.1.8 Troubleshooting Common Issues

- If cloning fails, check the network connection and repository URL.
- If GitHub authentication fails, use HTTPS with a token or configure SSH keys.
- If the local copy is incomplete, delete the folder and clone again.

3.2 Prerequisites

EngageMind depends on a small set of local services and development tools. MongoDB stores application data, Redis supports background training jobs, Node.js runs the frontend and auth backend, and Python runs the RAG and fine-tune services.

3.2.1 MongoDB (Database)

EngageMind uses **MongoDB** [5] for user profiles, uploaded documents, chat history, conversation state, and fine-tune metadata.

Setup Instructions

You can either:

- Install MongoDB locally: [MongoDB Community Edition](#)
- Use a managed cluster via [MongoDB Atlas](#)

Environment Configuration

Each service uses a separate environment variable name and, by default, a separate database:

```
1 # engagemind-backend/.env (auth service)
2 MONGO_URI=mongodb://localhost:27017/engagemindbackend
3
4 # engagemind-rag/.env (RAG + fine-tune service)
5 MONGO_URL=mongodb://localhost:27017/demo_db
```

Both variables can point to the same MongoDB instance but different databases, or to a hosted instance by replacing the URI accordingly.

MongoDB Collections

The following collections are used:

Collection	Description
users	Stores auth credentials, profile info (auth backend)
documents	Uploaded file metadata and binary contents (RAG backend)
chats	Chat sessions with all messages per user (RAG backend)
conversation_states	Serialised LangGraph memory state per thread (RAG backend)
models	Fine-tune task metadata and artifact paths (RAG backend)

Table 3.1: MongoDB collections used by EngageMind services.

3.2.2 Flask Backend (RAG + Inference Engine)

The **Flask-based Python backend** [6] handles ingestion, retrieval, memory, and response generation using LangChain, LangGraph, FAISS, and GPT/Mistral APIs.

Python Version

- Python 3.10 or higher recommended.

Installing Dependencies

Navigate to the backend directory and create a virtual environment:

```
1 python3 -m venv .venv
2 source .venv/bin/activate # On Windows: .venv\Scripts\activate
3 pip install -r requirements.txt
```

This installs:

- Flask (API server)
- LangChain (retrieval) [7]
- LangGraph (memory) [8]
- FAISS (vector similarity) [9]
- PyTorch (for GPT-2 fine-tuning) [10]

Key Features

- **Document ingestion pipeline:** Accepts multiple file types and chunks them for vector storage.
- **LangChain integration:** Handles context-aware RAG with memory recall.
- **FAISS search engine:** Enables fast similarity queries over embedded data.

- **Response delivery:** Chat responses are returned through authenticated REST API calls.

3.2.3 Express Backend (User Authentication)

The **Express backend** [11] is responsible for secure authentication and profile handling.

Node Requirements

- Node.js v18+ and npm. Install from Node.js official site. [12]

Installing Dependencies

Inside the `engagemind-backend/` directory:

```
1 npm install
```

This sets up:

- Express (server)
- Passport.js (OAuth strategies) [13]
- JWT (token management) [14]
- Resend (email API) [15]

Auth Features

- **JWT Auth:** Email/password registration and login flow with secure tokens.
- **Google OAuth:** Integrated via Passport.js, handles social sign-ins. [16]
- **Resend Email API:**
 - Sends verification emails after registration.
 - Provides password reset via emailed links.
- **Profile APIs:** Allow users to edit name, avatar, bio, and view history.

3.2.4 React Frontend

The frontend is built using **React** [17] (Create React App toolchain) and styled using **Tailwind CSS** [18].

Features

- **Authentication UI:**
 - Login and signup pages.
 - Google OAuth button.
 - Forgot password + Resend verification.
- **Chat Interface:**
 - Input + response threads.
 - File uploader.
 - Fine-tune training controls and status panel.
- **Profile Pages:**
 - Edit profile info (name, bio, avatar).
- **Controlled Error UI:**
 - Graceful fallback UIs for route access errors and missing data.

Setup Instructions

Navigate to the frontend folder and run:

```
1 npm install
2 npm start
```

Use `.env` to define API endpoints for the backend.

3.2.5 Background Services: Celery and Redis

These services enable asynchronous fine-tune job execution and status tracking.

Redis

Redis [\[19\]](#) is used as a **message broker** between Flask and Celery.

- Required for task queue management. Install via:

```
1 sudo apt install redis-server
```

- Running Redis locally on port 6379.

Celery

Celery [20] handles asynchronous jobs like:

- Queueing GPT-2 LoRA fine-tune training jobs
- Running long training tasks outside the request/response cycle
- Reporting task states (PENDING, PROGRESS, SUCCESS, FAILURE) to status endpoints

Start the worker inside the Flask directory with:

```
1 celery -A fine_tune.celery_config.app worker --pool=solo --  
    concurrency=1 --loglevel=info
```

Ensure Redis is running before launching the Celery worker.

Following these setup steps ensures all core EngageMind services are properly initialized for development and testing.

3.3 External APIs Used

EngageMind uses external services only where the local code needs hosted infrastructure: Mistral for embeddings and chat generation, Tavily as an optional web fallback, Resend for transactional email, and Google OAuth for social login.

3.3.1 Mistral API

Role:

- Generates embeddings for uploaded documents and user queries.
- Provides the default chat model when `CHAT_PROVIDER=mistral`.

Setup:

- Register at the Mistral API Platform [21].
- Generate your API key and store in `.env`:

```
1 MISTRAL_API_KEY=your_mistral_api_key
```

Integration:

- Used in the Flask backend through LangChain/Mistral SDK integrations for embeddings and default chat generation.
- Required when:

- Generating embeddings for uploaded documents and queries.
- Using the default `CHAT_PROVIDER=mistral` mode.
- Representative usage pattern:

```
1 embeddings = MistralAIEmbeddings(  
2     model="mistral-embed",  
3     mistral_api_key=MISTRAL_API_KEY  
4 )
```

3.3.2 Tavily Web Search API

Purpose: Provides web search results only when configured and when the graph needs fallback context.

Role:

- Not required for the normal document-upload and retrieval path.
- Used as an optional fallback when internal retrieval produces no usable context inside the LangGraph workflow.

Setup:

- Sign up at Tavily API Platform [\[22\]](#).
- Add to your environment variables:

```
1 TAVILY_API_KEY=your_tavily_api_key
```

Integration:

- Configured only when `TAVILY_API_KEY` is present and the Tavily package is available.
- Used as an optional LangGraph fallback when retrieval produces no usable context inside the graph.
- If the user has no uploaded documents at all, the current message handler returns guided upload advice before the graph runs.

3.3.3 Resend API (Email Delivery)

Purpose: Handles transactional emails such as:

- Verification email on registration
- Password reset emails

Role:

- Sends verification links for local account registration.
- Sends password reset links for account recovery.

Setup:

- Sign up at Resend Email API Service [15].
- Add to your `.env` in the Express auth service:

```
1 RESEND_API_KEY=your_resend_api_key
```

Integration:

- Password reset:
 - Generates a secure link with a token.
 - Sends it using Resend's transactional email API.
- Email verification:
 - Triggered after user registration.
 - Token is saved in DB and validated on confirmation.
- Sample call (Node.js):

```
1 await resend.emails.send({
2   from: process.env.RESEND_FROM_EMAIL,
3   to: user.email,
4   subject: "Reset your password",
5   html: '<a href="${resetUrl}">Click here to reset</a>',
6 });
```

3.3.4 Google OAuth (Passport.js)

Purpose: Allows users to sign up and log in using their Google account.

Role:

- Allows users to log in with a Google account.
- Requires `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, and the configured callback route.

Setup:

- Create OAuth credentials via Google Cloud Console [23].
- Add the following to your `.env` in the auth server:

```
1 GOOGLE_CLIENT_ID=your_google_client_id
2 GOOGLE_CLIENT_SECRET=your_google_client_secret
```

Integration:

- Passport.js strategy configured for Google.
- Tokens exchanged and user stored or retrieved from DB.
- Integrated into frontend login flow with a Google button.

3.4 Software Architecture

3.4.1 System Overview

EngageMind is a full-stack conversational application built around four local services. The frontend gives the user one place to register, log in, chat, upload documents, and start a training job. The backend services keep those concerns separate: authentication is handled by Express, RAG chat by Flask, and GPT-2 LoRA training by a second Flask API with a Celery worker. The main capabilities are:

- document ingestion for text, Markdown, PDF, and DOCX files,
- per-user conversation history and LangGraph state,
- authentication through email/password or Google OAuth,
- document-grounded answers through FAISS retrieval,
- GPT-2 LoRA training jobs with status polling and optional GPT-2 LoRA chat mode.

The system is split into four runtime services:

1. **Frontend (React):** A responsive UI for user interaction, served on port 3000
2. **Auth Backend (Node.js + Express):** Handles authentication, user profiles, and email workflows, served on port 5003
3. **RAG Backend (Python + Flask):** Manages document ingestion, retrieval, and conversational AI logic, served on port 5001
4. **Fine-Tune API (Python + Flask + Celery):** Orchestrates asynchronous GPT-2 LoRA training on user documents, served on port 5002

MongoDB stores users, uploaded documents, conversations, graph state, and training metadata. FAISS stores the vector index for each user separately, so retrieval is scoped to that user's uploaded material.

3.4.2 Architecture Diagram

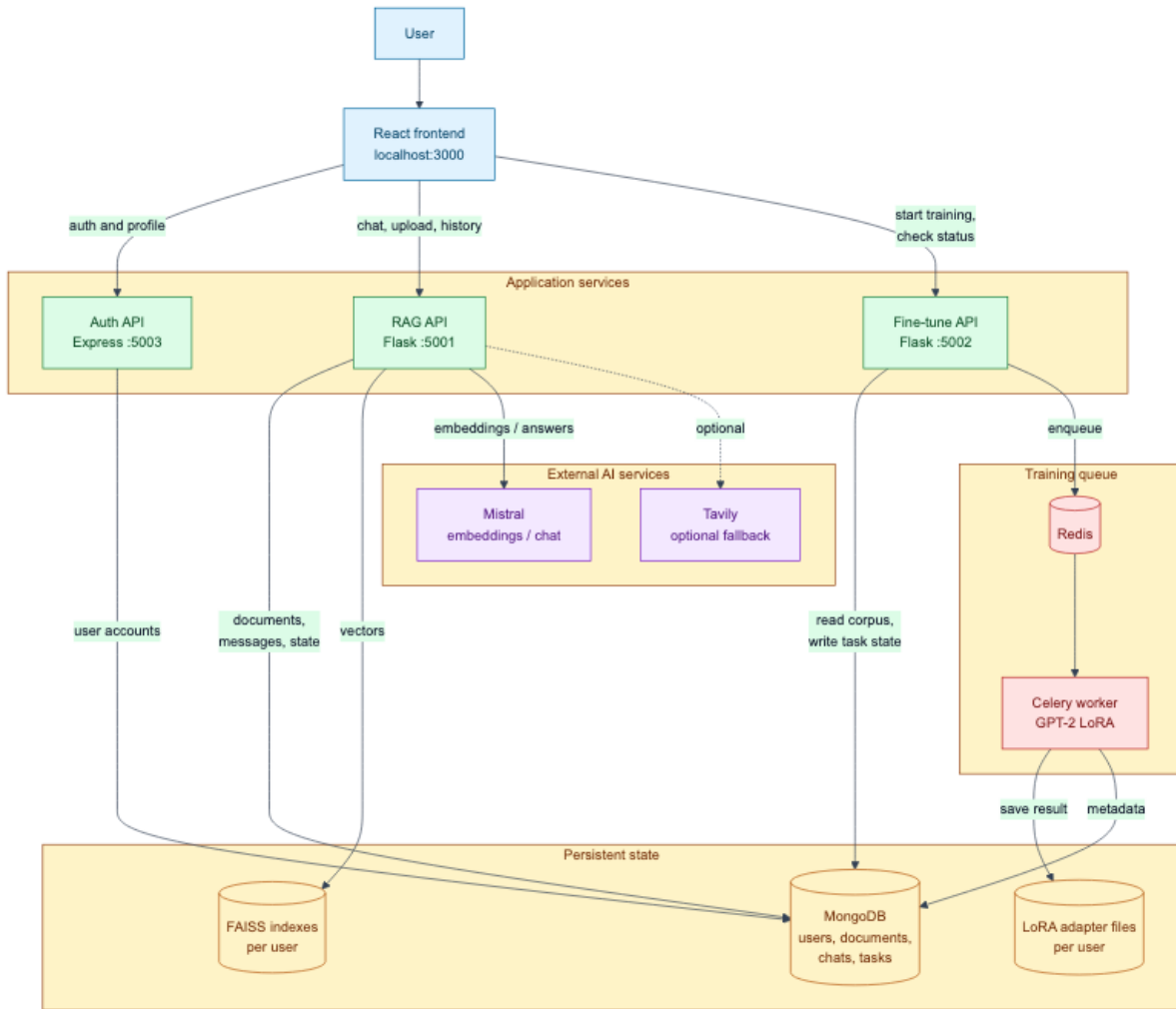


Figure 3.1: EngageMind System Architecture

3.4.3 System Logic Overview

The services communicate through REST APIs. The frontend sends authentication requests to the Express backend, chat and upload requests to the RAG API, and training plus GPT-2 LoRA availability/chat requests to the fine-tune API. The main workflows are:

- **Document Upload:**
 - The React frontend sends a file to `/api/upload`.
 - Flask stores the file in MongoDB and updates the user's FAISS index.
 - If an incremental update fails, the index is rebuilt from stored documents.
- **Chat Interaction:**

- The frontend sends a message to `/api/conversation/<conversation_id>/message`.
- Flask loads the conversation and invokes the LangGraph workflow, which:
 1. Fetches relevant context using query rewriting and FAISS
 2. generates an answer from the retrieved context,
 3. optionally checks grounding and relevance,
 4. saves the assistant message and updated state.
- Returns response to frontend
- **Memory Management:**
 - Conversation state is stored in MongoDB (`conversation_states`).
 - LangGraph state is serialized as Python dictionaries so it can be loaded on the next turn.
- **GPT-2 LoRA Chat Mode:**
 - The frontend checks adapter availability with `/api/fine-tune/model`.
 - When available, the user switches mode and sends messages to `/api/fine-tune/chat`.
 - The fine-tune API loads the per-user adapter, generates a response, and persists messages to the same chat collection.

3.4.4 RAG System Details

Document Ingestion (`ingestion_pipeline.py`)

The ingestion pipeline turns uploaded files into searchable text chunks:

- **Supported Formats:** `.txt`, `.md`, `.pdf`, and `.docx`
- **Chunking:** Text segmented using `RecursiveCharacterTextSplitter` for optimal embedding
- **Embedding:** Generates vectors with `MistralAIEmbeddings`
- **Indexing:** Constructs a FAISS vector store, saved to `faiss_index_dir/{user_id}` for per-user isolation

Retrieval Logic (`retrieval_pipeline.py`)

The retrieval pipeline loads the current user's index and retrieves context for the question:

- Loads user-specific FAISS index from disk

- Uses a **rewriting retriever** powered by LangChain, reformulating queries via a Mistral-based LLM prompt
- Returns top-k documents based on semantic similarity

3.4.5 LangGraph Memory Workflow (`app.py`)

Conversations use a **LangGraph** workflow with two key nodes:

1. `retrieve__context`:

- Loads conversation history from MongoDB
- Rewrites the user query for enhanced retrieval
- Executes the retrieval chain to fetch relevant documents
- Updates the conversation's memory state

2. `generate__response`:

- Builds a prompt with context and chat history
- Invokes the Mistral LLM for response generation
- Performs hallucination and relevance grading using LangChain
- Appends a disclaimer if the response fails grounding or relevance checks
- Saves the updated conversation state

Memory Persistence:

- Stored in the `conversation_states` MongoDB collection
- State is a serialized Python `dict` containing:
 - `user_id`, message history, retrieved context (documents), and web fallback flag

3.4.6 Authentication System

Express + MongoDB (`server.js`, `userModel.js`)

The authentication service handles accounts and protected profile routes:

- **Local Auth**: Email/password registration and login
- **Google OAuth**: Seamless social login via **Passport.js**
- **Email Workflows**: Verification and password reset emails sent via **Resend API**
- **Token Management**: JWTs issued upon login, stored client-side

MongoDB User Schema

```

1 {
2   email, password, provider: 'local' | 'google',
3   googleId, verified,
4   avatar, bio, socialLinks, role,
5   verificationToken, passwordResetToken
6 }

```

3.4.7 Database Design (MongoDB)

MongoDB collections are used as follows:

Collection	Purpose
users	Authentication and profile information
documents	Ingested documents with binary content
chats	Chat sessions with all messages
conversation_states	Serialized LangGraph memory state

Table 3.2: MongoDB Collections

Each user has:

- A dedicated FAISS index stored on disk
- Multiple chat threads tracked in `chats`
- Per-message state persistence in `conversation_states`

3.4.8 API Surface

Flask `/api/`

Express `/auth/`

3.4.9 Status Feedback & Evaluation

- **Response Delivery:** Assistant responses are returned through REST APIs; the current implementation does not use WebSockets.
- **Evaluation:** LangChain-based grading assesses:
 - **Grounding:** Is the response supported by retrieved documents?
 - **Relevance:** Does the response address the user’s query?
- **Failure Tracking:** Responses that fail quality checks are logged for review.

Endpoint	Method	Description
/api/health	GET	Service health and dependency status (public)
/api/upload	POST	Uploads document and updates per-user index
/api/ask	POST	Legacy single-question endpoint
/api/conversation	POST	Creates a new conversation thread
/api/conversation/<conversation_id>/message	POST	Sends message and returns AI response
/api/conversation/<conversation_id>	GET	Loads full conversation
/api/conversations	GET	Lists all user threads
/api/conversation/<conversation_id>	DELETE	Deletes thread
/api/index/stats	GET	Returns index statistics for current user

Table 3.3: Flask API Endpoints

3.4.10 Deployment Considerations

- **Local Development:** `scripts/run_all.sh` starts the frontend, auth backend, RAG API, fine-tune API, and Celery worker when dependencies are available.
- **Storage:** MongoDB and the FAISS index directory must be preserved between runs.
- **Rate Limiting:** Flask-Limiter protects the RAG API from excessive requests.
- **Configuration:** Each service reads its own `.env` file; backend and RAG must share the same `JWT_SECRET`.

3.5 RAG Implementation

The RAG backend is implemented as a modular Flask service that combines document ingestion, FAISS-based retrieval, LangGraph conversation state, and quality checks. Uploaded documents are stored in MongoDB, parsed into text, chunked, embedded with Mistral embeddings, and saved in a per-user FAISS index. During chat, the authenticated user's message is routed through intent detection, optional query rewriting, retrieval, response generation, and persistence back to MongoDB.

- **Ingestion:** `process_text_content`, `process_binary_content`, and `build_faiss_index` convert uploaded `.txt`, `.md`, `.pdf`, and `.docx` documents into searchable chunks.
- **Retrieval:** `build_retrieval_chain` loads the user's FAISS index and returns relevant chunks for grounded answers.

Endpoint	Method	Description
/auth/register	POST	Registers user
/auth/login	POST	Logs in and returns JWT
/auth/verify-email	GET	Verifies email token
/auth/google	GET	Google OAuth redirect
/auth/forgot-password	POST	Sends password reset link
/auth/reset-password	POST	Resets password with token
/auth/profile	GET	Returns authenticated user profile
/auth/edit-profile	PUT	Updates name, bio, avatar
/admin/protected	GET	Protected route example
/admin/admin-data	GET	Admin-only route example

Table 3.4: Express API Endpoints

- **Memory:** `save_state` and `load_state` persist LangGraph state in MongoDB using a user-and-conversation thread key.
- **Quality:** the evaluation module can rewrite queries and grade hallucination/relevance when quality checks are enabled.
- **Fallback:** if a user has no uploaded documents, the message handler returns guided upload advice; optional Tavily fallback is available inside the graph when configured.

This condensed implementation view preserves the key technical behavior without repeating the architecture details already introduced earlier in the chapter.

3.6 UserAuth Backend Implementation

The authentication backend is a Node.js/Express service. It owns local registration, login, email verification, password reset, Google OAuth, profile updates, and admin-protected examples. MongoDB stores user records through Mongoose, JWT tokens protect private routes, Passport.js handles Google OAuth, and Resend sends emails when the required API key is configured.

Key functionalities include:

- **Authentication:** Supports local email/password and Google OAuth via Passport.js, with JWT-based session management.
- **Email Verification:** Creates and validates verification tokens for new local accounts, sent via Resend API when configured.
- **Password Recovery:** Enables secure password resets through time-bound email links.
- **Profile Management:** Allows users to update bio, avatar, and social links.
- **Access Control:** Enforces user/admin role checks for restricted endpoints.

Project Structure

The backend is organized as follows:

- `config/`: Database and Passport.js configurations.
- `middleware/`: Authentication and role-based access middleware.
- `models/`: Mongoose schema for user data.
- `routes/`: API endpoint definitions.
- `server.js`: Main server file.
- `.env`: Environment variables (e.g., `MONGO_URI`, `JWT_SECRET`, `GOOGLE_CLIENT_ID`, `RESEND_API_KEY`).

Configuration

- **Database**: Connects to MongoDB via Mongoose, using `MONGO_URI` from environment variables (`config/db.js`).
- **Authentication**: Configures Passport.js with JWT and Google OAuth strategies (`config/passport.js`). JWT validates protected routes, while Google OAuth creates or logs in users based on `googleId`.

User Model

The `User` schema (`models/userModel.js`) defines: Timestamps (`createdAt`, `updatedAt`) are automatically added.

Middleware

- **Authentication** (`middleware/authMiddleware.js`): Verifies JWT tokens, attaching the user to the request.
- **Role-Based** (`middleware/roleMiddleware.js`): Restricts access to admin-only endpoints, returning 403 if unauthorized.

API Endpoints

Application Flow

- **Registration**: Users submit credentials, receive a verification token/email flow, and are stored with `role: user`.

Field	Type	Description
username	String	Unique username, required
email	String	Unique email, required
password	String	Required for local accounts
provider	String (enum)	local or google
googleId	String	Google OAuth ID (optional)
role	String (enum)	user (default) or admin
verified	Boolean	Email verification status
bio	String	User biography
avatar	String	Profile picture URL
socialLinks	Object	Social media links (Twitter, LinkedIn, GitHub)
verificationToken	String	Email verification token
passwordResetToken	String	Password reset token

Table 3.5: User Model Schema

- **Google Login:** Users authenticate via Google OAuth, with accounts created or logged in based on googleId.
- **Password Reset:** Users request a reset link, sent via Resend API, and submit a new password using a time-bound token.
- **Protected Actions:** JWT tokens secure access, with admin endpoints restricted to role: admin.

Error Handling

- **Structured Responses:** APIs return appropriate HTTP codes (e.g., 401 for invalid JWT, 403 for forbidden access).
- **Validation:** Failures (e.g., duplicate emails) yield 400 Bad Request responses with clear messages.
- **Logging:** Errors are logged for debugging and monitoring.

3.6.1 User Authentication Backend Summary

The authentication backend keeps user identity separate from the RAG service. It creates and verifies accounts, issues JWT tokens, supports Google OAuth when configured, and exposes profile endpoints used by the frontend.

Key components include:

Endpoint	Method	Description	Auth Required
/auth/register	POST	Register user, send verification email	No
/auth/login	POST	Authenticate user, issue JWT	No
/auth/google	GET	Redirect to Google OAuth	No
/auth/google/callback	GET	Handle Google OAuth callback	No
/auth/forgot-password	POST	Send password reset link	No
/auth/reset-password	POST	Reset password with token	No
/auth/verify-email	GET	Verify email address	No
/auth/profile	GET	Get authenticated user profile	Yes
/auth/edit-profile	PUT	Update profile fields	Yes
/admin/protected	GET	Protected route example	Yes
/admin/admin-data	GET	Fetch admin data (admin only)	Yes (admin)

Table 3.6: Authentication Backend API Endpoints

- **Authentication:** Supports local email/password and Google OAuth via Passport.js, with JWT tokens securing sessions.
- **Email Verification:** Provides a verification-token flow for local accounts using Resend API when configured.
- **Password Recovery:** Provides time-bound password reset links sent via email.
- **Profile Management:** Allows updates to bio, avatar, and social links, stored in MongoDB.
- **Access Control:** Restricts admin endpoints to users with `role: admin`.

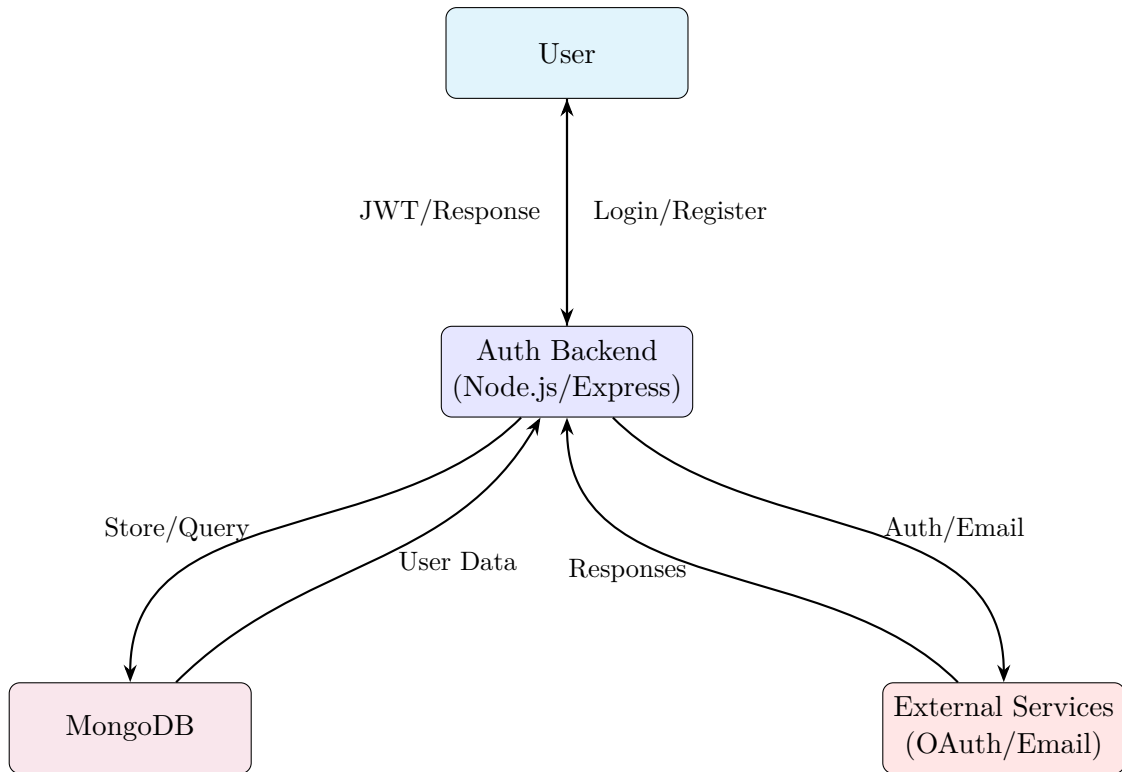


Figure 3.2: Simplified Flow Diagram of EngageMind Authentication Backend

3.7 Fine-Tune API Implementation

The Fine-Tune API is a dedicated Flask service (port 5002) that enables users to train a GPT-2 model adapted to their own uploaded documents using LoRA (Low-Rank Adaptation). Training is executed asynchronously via Celery workers, allowing the frontend to remain responsive while a long-running training job is in progress.

3.7.1 Service Overview

- **Framework:** Flask (Python)
- **Task Queue:** Celery with Redis as the message broker
- **Base Model:** GPT-2 (via Hugging Face `transformers`)
- **Adaptation:** LoRA via the PEFT library (`r=16`, `lora_alpha=32`)
- **Port:** 5002

3.7.2 API Endpoints

All endpoints require a valid JWT (`Authorization: Bearer <token>`) to identify the requesting user.

Endpoint	Method	Description
/api/fine-tune	POST	Extracts user document corpus and queues a Celery training task. Returns a <code>task_id</code> .
/api/fine-tune/status/<task_id>	GET	Returns current task status: <code>PENDING</code> , <code>PROGRESS</code> , <code>SUCCESS</code> , or <code>FAILURE</code> .
/api/fine-tune/model	GET	Returns whether the authenticated user has a usable GPT-2 LoRA adapter.
/api/fine-tune/chat	POST	Generates a response using the user's GPT-2 LoRA adapter (message + optional <code>conversation_id</code>).

Table 3.7: Fine-Tune API Endpoints

3.7.3 Training Workflow

1. **Corpus Extraction:** User documents are fetched from the MongoDB `documents` collection and decoded from binary.
2. **Tokenisation:** Texts are tokenised with GPT-2's tokeniser at a maximum length of 256 tokens.
3. **LoRA Setup:** A LoRA configuration is applied to the GPT-2 base model using PEFT.
4. **Training:** One epoch of training runs with batch size 1, suitable for CPU or low-memory environments.
5. **Artifact Saving:** The adapter weights and tokeniser are saved to `models/{user_id}/gpt2-lora/` and metadata is written to the MongoDB `models` collection.

3.7.4 Frontend Integration

The React frontend polls the status endpoint via `axiosFineTune` (base URL: `http://localhost:5002`) using the functions `startFineTuneTraining()`, `fetchFineTuneStatus(taskId)`, `fetchFineTuneModelStatus()`, and `sendFineTuneMessage()` defined in `chatApi.jsx`. The `Sidebar` component renders the current training status and updates automatically as the task progresses. The `ChatWindow` component exposes a mode selector that routes messages to either the RAG API or the GPT-2 LoRA chat endpoint when an adapter is available.

3.8 Frontend Implementation

3.8.1 Project Overview

The frontend is a React application built around a few practical flows: registration and login, protected chat, document upload, fine-tune status, and profile editing. API calls are split by service: `axiosAuth` talks to the Express backend, `axiosChat` talks to the RAG API, and `axiosFineTune` talks to the fine-tune API.

3.8.2 Application Bootstrapping: `index.js`

The application initializes in `index.js`, where the root component `<App />` is rendered into the DOM element identified by `root`. Styles are applied through `index.css`, which integrates TailwindCSS and custom styles for consistent presentation.

```
1 import './index.css';
2 import App from './App';
3
4 const root = ReactDOM.createRoot(document.getElementById('root'));
5 root.render(
6   <React.StrictMode>
7     <App />
8   </React.StrictMode>
9 );
```

3.8.3 Routing Architecture: `App.js`

Routes are defined in `App.js`. Public pages are available to all visitors, while chat and profile pages require a token in `localStorage`.

3.8.4 User Authentication Flow

The frontend keeps the JWT token in `localStorage`. That token is used for route access and for authenticated API calls.

Authentication Token Behavior

- **Key:** token
- **Storage:** Stored in `localStorage` after successful login (or Google OAuth callback).
- **Usage:**
 - Attached to every authenticated API request via Axios interceptors in the `Authorization` header.

Type	Path	Component	Description
Public	/	HomePage	Landing page for all users
Public	/login	LoginPage	User login interface
Public	/register	RegisterPage	User registration form
Public	/reset-password	ResetPasswordPage	Reset password form
Public	/forgot-password	ForgotPasswordPage	Password-reset request form
Public	/verify-email	VerifyEmailPage	Email verification page
Public	/auth-success	AuthSuccess	Google OAuth success redirect page
Public	/access-denied	AccessDeniedCard	Restricted-access notice
Protected	/chat	ChatPage	Secure chat interface
Protected	/profile	UserProfile	User profile viewing page
Protected	/edit-profile	EditProfile	Profile editing interface
Fallback	*	Redirect to /	Redirects undefined routes to home

Table 3.8: Routing Architecture

- Checked by protected routes to determine access eligibility.

AxiosAuth Configuration

The Axios instance for authentication, defined in `/src/api/axiosAuth.js`, adds the authentication token only when it exists in `localStorage`. The base URL for authentication-related APIs is `http://localhost:5003`.

```

1 const token = localStorage.getItem('token');
2 if (token) {
3   config.headers.Authorization = `Bearer ${token}`;
4 }

```

- **Interceptor Behavior:** The interceptor adds `Authorization: Bearer <token>` only when a token exists in `localStorage`.

Authentication Pages and Step-by-Step Flow

- **LoginPage:**

1. User enters credentials (e.g., email and password).
2. Form submission triggers an API call: `axiosAuth.post('/auth/login', { emailOrUsername, password })`.

3. On success, the backend returns a JSON response containing the **token**.
 4. Token is saved: `localStorage.setItem('token', response.data.token)`.
 5. User is redirected to `/chat` after successful authentication.
 6. **Error Handling**: If credentials are invalid, an error message is displayed via the `ErrorDisplay` component.
- **RegisterPage**:
 1. User submits registration data (e.g., username, email, password).
 2. API call: `axiosAuth.post('/auth/register', { username, email, password })`.
 3. Backend initiates email verification by sending a confirmation link.
 4. The response includes a verification token used by the frontend verification page flow.
 5. User is redirected to `/verify-email?token=...` and then to login after verification.
 6. **Error Handling**: Duplicate email or weak password triggers an error displayed to the user.
 - **ForgotPasswordPage**:
 1. User enters their email address.
 2. API call: `axiosAuth.post('/auth/forgot-password', { email })`.
 3. Backend sends a password reset link to the provided email.
 4. Success message is shown; no token is involved at this stage.
 5. **Error Handling**: Invalid email results in an error notification.
 - **ResetPasswordPage**:
 1. Accessed via a tokenized URL (e.g., `/reset-password?token=xyz`).
 2. User enters a new password.
 3. API call: `axiosAuth.post('/auth/reset-password', { token, newPassword })`.
 4. On success, user is redirected to `/login`.
 5. **Error Handling**: Expired or invalid token displays an error.
 - **VerifyEmailPage**:
 1. Accessed via a verification URL (e.g., `/verify-email?token=abc`).
 2. API call: `axiosAuth.get('/auth/verify-email?token=abc')`.
 3. Backend confirms the email and returns a success/error message.
 4. Frontend redirects to `/login` after successful verification.
 5. **Error Handling**: Invalid token shows an error message.

3.8.5 ProtectedRoute Logic

The `ProtectedRoute` component checks whether the browser has a token before rendering protected pages.

```
1 const isAuthenticated = !!localStorage.getItem('token');
```

Detailed Behavior

- **Token Check:**
 - The condition `!!localStorage.getItem('token')` evaluates to `true` if a token exists, `false` otherwise.
- **Access Scenarios:**
 - **Authenticated User:** If `isAuthenticated` is `true`, the requested component (e.g., `ChatPage`) is rendered.
 - **Unauthenticated, Accessing `/chat`:** If `isAuthenticated` is `false`, the `AccessDeniedCard` is displayed with a registration link (`/register`).
 - **Unauthenticated, Other Protected Routes:** Redirects to `/login` using React Router's navigation.
- **Loading State:** During initial data fetching (e.g., conversations for `/chat`), a `LoadingSpinner` is shown.
- **Error Handling:** If data fetching fails, the `ErrorDisplay` component renders with the error details.

3.8.6 Chat System Detailed Flow

The chat system enables users to create conversations, send messages, and upload documents, all authenticated via the same `token` used elsewhere. It relies on the `axiosChat` API instance.

LocalStorage Behavior

- **Key:** `token` (shared with authentication).
- **Usage:** Axios interceptors attach `Authorization: Bearer <token>` to all chat-related requests.

axiosChat Configuration

Defined in `/src/api/axiosChat.js` with a base URL of `http://localhost:5001`.

```
1 const token = localStorage.getItem('token');  
2 config.headers.Authorization = `Bearer ${token}`;
```

- **Interceptor:** Ensures every request includes the token, failing which the request is rejected by the backend with a 401 status.

Chat Functionalities

Fetch Conversations

- **Function:** `fetchConversations()`
- **API Call:** `axiosChat.get('/api/conversations')`
- **Step-by-Step:**
 1. Triggered on `ChatPage` load if authenticated.
 2. Request sent with `Authorization` header.
 3. Backend returns an array of conversation summaries (e.g., `{id, title, lastMessage}`).
 4. Data populates `Sidebar` and `ChatList` components.
- **Error Handling:** Network failure or 401 status displays `ErrorDisplay`.

Create New Conversation

- **Function:** `createConversation()`
- **API Call:** `axiosChat.post('/api/conversation')`
- **Step-by-Step:**
 1. User clicks a "New Chat" button in `Sidebar`.
 2. POST request sent with `Authorization` header.
 3. Backend responds with a new `conversation_id`.
 4. Frontend updates `ChatList` with the new conversation.
- **Error Handling:** Failure (e.g., server error) shows an error message.

Fetch Conversation Messages

- **Function:** `fetchConversationMessages(conversationId)`
- **API Call:** `axiosChat.get('/api/conversation/${conversationId}')`

- **Step-by-Step:**
 1. User selects a conversation from `ChatList`.
 2. GET request includes the specific `conversationId`.
 3. Backend returns message history (e.g., `{id, sender, content, timestamp}`).
 4. Messages render in `ChatWindow`.
- **Error Handling:** Invalid `conversationId` or 404 triggers `ErrorDisplay`.

Send a Message

- **Function:** `sendMessage(conversationId, message)`
- **API Call:** `axiosChat.post('/api/conversation/${conversationId}/message', { message })`
- **Step-by-Step:**
 1. User types a message in `ChatWindow` and submits.
 2. POST request sends the message payload.
 3. Backend processes and returns a bot response.
 4. Both user message and bot reply update `ChatWindow`.
- **Error Handling:** Failed send displays an error; message remains unsent.

Ask a Question (Outside Conversation)

- **Function:** `askQuestion(question)`
- **API Call:** `axiosChat.post('/api/ask', { question })`
- **Step-by-Step:**
 1. User submits a question via a global input (e.g., in `Navbar`).
 2. POST request sent without a `conversationId`.
 3. Backend provides an immediate response.
 4. Response displayed in a temporary UI element.
- **Error Handling:** Server error shows a notification.

Upload a Document

- **Function:** `uploadDocument(file)`

- **API Call:** `axiosChat.post('/api/upload', formData, { headers: {'Content-Type': 'multipart/form-data'}})`
- **Step-by-Step:**
 1. User selects a file in `DocumentUploader`.
 2. File is packaged into a `FormData` object.
 3. POST request sent with custom headers and **Authorization**.
 4. Backend stores the file and returns a confirmation.
 5. UI updates to reflect upload success.
- **Error Handling:** File size limits or type errors trigger `ErrorDisplay`.

3.8.7 Components Connection

The application comprises interconnected components, each with a specific role:

Component	Purpose
Navbar	Header navigation bar with links and user options
Footer	Static page footer with additional information
Layout	Structural wrapper for public-facing pages
LoadingSpinner	Visual indicator for ongoing operations
ErrorDisplay	Displays error messages with details
AccessDeniedCard	Warns of unauthorized access with registration link
ChatPage	Main interface for chat functionalities
ChatContainer	Combines sidebar and chat window layout
Sidebar	Displays list of user conversations
ChatList	Renders conversation summaries
ChatWindow	Real-time chat message display and input
DocumentUploader	Interface for file uploads in chats
LoginPage, RegisterPage	Authentication entry points
UserProfile, EditProfile	Profile viewing and editing pages

Table 3.9: Main Components and Their Purposes

3.8.8 Loading and Error Handling

Loading and error states are managed consistently across the application:

- **Initial Load:** If authenticated, `App.js` calls `fetchConversations()` to preload chat data.
- **Loading State:** `LoadingSpinner` appears during API calls.
- **Error State:**

```
1 if (error) return <ErrorDisplay error={error} />;
```

- Errors from API calls (e.g., 401, 404) are caught and rendered with custom messages.

3.8.9 LocalStorage Key List

The application utilizes the following `localStorage` keys:

Key	Purpose
<code>token</code>	Primary authentication token for user sessions

Table 3.10: LocalStorage Keys

3.8.10 Data Flow Diagram

The application's data flow is outlined below in a textual representation:

```
User -> (Login/Register) -> token saved -> Routes open
-> Access Chat -> fetchConversations -> Sidebar -> Select conversation
    -> fetchConversationMessages -> ChatWindow
    -> sendMessage -> Chat updated
-> (Optionally) uploadDocument -> enrich chat context
-> Profile -> View/Edit profile
```

3.8.11 Chat Application Flow

The chat flow starts after authentication. The frontend loads the user's conversations, lets the user open or create a conversation, sends messages to the RAG API, uploads documents through `/api/upload`, and polls fine-tune status when a training job is active. Profile pages use the auth backend separately.

3.9 Concrete Improvements

EngageMind now has the main thesis features in place: login, persistent conversations, document upload, RAG chat, GPT-2 LoRA training, task status polling, and GPT-2 LoRA live chat through the saved adapter. That makes the system demo-ready. It does not make it finished. The next work should be narrow and practical, because the weak points are easy to name.

3.9.1 Harden GPT-2 LoRA Live Chat

The fine-tune service trains GPT-2 with LoRA, saves the adapter under `engagemind-rag/models/{user_id}/gpt2-lora/`, and exposes two protected endpoints for adapter-based chat:

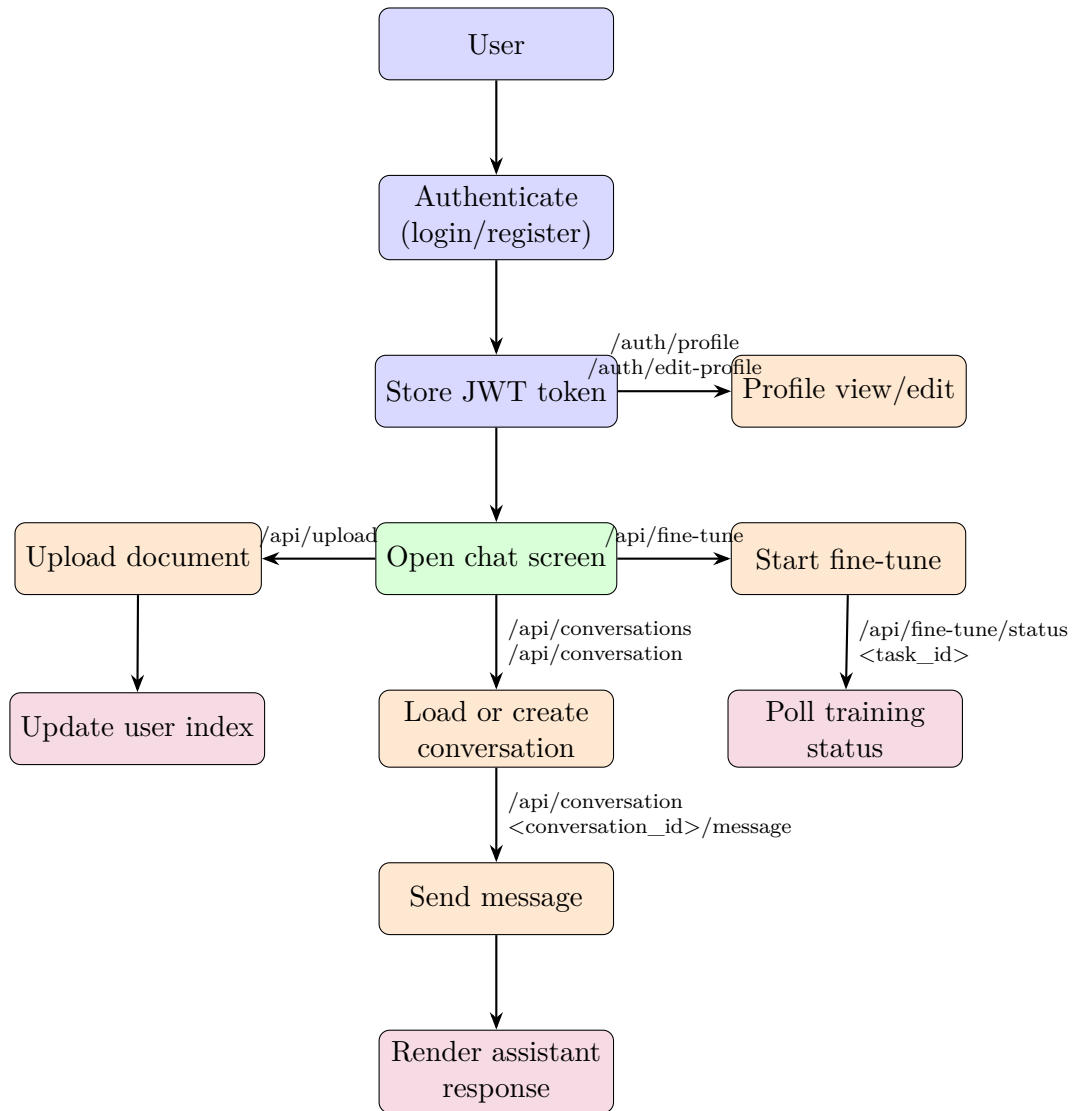


Figure 3.3: Chat System Flow

```
1 GET    /api/fine-tune/model
2 POST   /api/fine-tune/chat
```

The first endpoint checks whether the signed-in user has a complete adapter and matching MongoDB metadata. The second endpoint loads GPT-2, attaches the user's adapter with PEFT, generates a response, and stores the exchange in the same `chats` collection used by RAG.

The next step is hardening, not another chat route. The adapter cache should have a fixed size, least-recently-used eviction, and clear logs for load time and memory use. Model-load failures should return a controlled response that tells the frontend the adapter could not be loaded. The current implementation already avoids the worst local memory problem by loading GPT-2 with a lower-memory dtype, but this should be tested on the exact laptop or server used for the demo.

3.9.2 Persist Training State Across Page Refresh

Training status still depends too much on the browser tab. If the user refreshes the page, the UI can lose the task id even though Celery still has the job. I would store one task record in MongoDB when `/api/fine-tune` queues training:

- `user_id`
- `task_id`
- `status`
- `started_at`
- `completed_at`
- `output_dir`

Then the frontend can call `GET /api/fine-tune/latest` after page load. The sidebar would show the last known task instead of resetting to an empty state. The existing `GET /api/fine-tune/model` endpoint answers a different question: whether an adapter is ready for chat. It does not recover a running task.

3.9.3 Keep Expanding the Production Test Layer

The project now has a stronger verification script. It runs backend API tests, frontend wiring checks, the React smoke test, a production frontend build, RAG endpoint contracts, fine-tune contracts, security checks, architecture checks, and Python compile checks. The tests cover the main paths a real user hits:

- Auth backend: registration, duplicate rejection, email verification, login, bad login, profile read, profile edit, protected route access, non-admin rejection, invalid verification token, invalid reset token, and password-reset privacy for unknown emails.
- RAG API: JWT enforcement, health response shape, conversation create/list/get/delete, cross-user isolation, message validation, chitchat persistence, no-document fallback persistence, upload behavior, index-stat scoping, and legacy empty-question handling.
- Fine-tune API: authentication, empty corpus failure, queue failure, task status mapping, ObjectId serialisation, model availability, missing adapter handling, empty chat message rejection, cross-user conversation rejection, mocked successful inference, generation failure handling, GPT-2 LoRA message persistence, and cleanup of repeated role markers in generated text.
- Frontend: service-specific Axios clients, token routing, RAG endpoint wiring, fine-tune start/status/model/chat wiring, polling behavior, terminal training states, upload entry point, document upload errors, and switching between RAG and GPT-2 LoRA.

This is the right split for a thesis project. Fast tests mock model downloads and external APIs. The heavy path should stay separate: run a small manual or nightly test that uploads a tiny document, queues training, waits for **SUCCESS**, switches to GPT-2 LoRA mode, and sends one real chat message. That test proves the expensive chain without making every commit wait on model work.

3.9.4 Make Document Ingestion More Honest About PDFs

The PDF path extracts embedded text. It does not do OCR. That is fine for normal PDFs, but scanned PDFs need a clear warning instead of a silent weak result. Each upload response should include:

- extracted character count,
- chunk count,
- parser used,
- warning when no usable text was found.

The frontend should show that result after upload. If OCR is added later, it should run as a separate optional worker step. Hiding OCR inside the normal upload route would make uploads slow and hard to debug.

3.9.5 Add a Startup Configuration Check

The app depends on matching `JWT_SECRET` values, MongoDB, Redis, provider API keys, writable model folders, and the selected chat provider. The `/api/health` route helps

after the RAG service starts. I would also add a command-line check that runs before the demo and prints a plain pass/fail table.

The check should verify:

- backend and RAG share the same JWT secret,
- MongoDB is reachable and expected collections can be written,
- Redis is reachable,
- Mistral or OpenRouter config matches `CHAT_PROVIDER`,
- model artifact directories are writable.

This would catch the boring failures first: wrong secret, dead Redis, missing API key, or a model directory the worker cannot write to.

3.9.6 Keep User Data Boundaries Explicit

The code keys conversations, uploaded documents, FAISS indexes, and training artifacts by `user_id`. Tests already cover conversation isolation and GPT-2 LoRA cross-user conversation rejection. I would keep adding the same kind of check anywhere an id crosses a service boundary: document ids, index stats, model artifacts, and task records. A personalized assistant cannot leak another user's context. That is not a nice-to-have; it is the safety rule.

3.9.7 Improve Model Lifecycle Management

GPT-2 LoRA inference has basic caching. Loading an adapter on every request is slow. Keeping every user's adapter in memory is worse. I would replace the simple cache with:

- maximum loaded adapters,
- least-recently-used eviction,
- explicit unload on failure,
- CPU/GPU memory logging,
- a fallback response when the model cannot load.

That makes the laptop demo more stable and gives the same code a path toward deployment.

3.9.8 Tighten the Demo Story

The final demo should keep the two chat modes separate:

- **RAG mode:** upload a document, ask grounded questions, show that answers use retrieved context.
- **GPT-2 LoRA mode:** train on a small user corpus, switch modes, show that the saved adapter can generate live responses.

That distinction matters. RAG answers from retrieved documents. LoRA changes the model behavior. EngageMind uses both, but they solve different problems.

Acknowledgements

Bibliography

- [1] Alec Radford et al. “Language models are unsupervised multitask learners”. In: *OpenAI Blog* 1.8 (2019).
- [2] *GitHub: Where the world builds software*. <https://github.com/>. Accessed: 2025-04-25. 2024.
- [3] *Git - Distributed Version Control System*. <https://git-scm.com/>. Accessed: 2025-04-25. 2024.
- [4] *Homebrew - The Missing Package Manager for macOS (or Linux)*. <https://brew.sh/>. Accessed: 2025-04-25. 2024.
- [5] *MongoDB - The Developer Data Platform*. <https://www.mongodb.com/>. Accessed: 2025-04-25. 2024.
- [6] *Flask - Python Web Framework*. <https://flask.palletsprojects.com/>. Accessed: 2025-04-25. 2024.
- [7] *LangChain - Building Applications with LLMs through Composability*. <https://www.langchain.com/>. Accessed: 2025-04-25. 2024.
- [8] *LangGraph - State Machines for LLM Applications*. <https://langchain-ai.github.io/langgraph/>. Accessed: 2025-04-25. 2024.
- [9] *FAISS - Facebook AI Similarity Search*. <https://faiss.ai/>. Accessed: 2025-04-25. 2024.
- [10] *PyTorch - An Open Source Machine Learning Framework*. <https://pytorch.org/>. Accessed: 2025-04-25. 2024.
- [11] *Express - Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com/>. Accessed: 2025-04-25. 2024.
- [12] *Node.js is an open source server environment*. <http://nodejs.org/>. Accessed: 2025-04-25. 2024.
- [13] *Passport.js - Simple, unobtrusive authentication for Node.js*. <http://www.passportjs.org/>. Accessed: 2025-04-25. 2024.
- [14] *JWT.io - JSON Web Tokens*. <https://jwt.io/>. Accessed: 2025-04-25. 2024.
- [15] *Resend - Email API for developers*. <https://resend.com/>. Accessed: 2025-04-25. 2024.

- [16] *Google OAuth 2.0 Authorization Framework*. <https://developers.google.com/identity/protocols/oauth2>. Accessed: 2025-04-25. 2024.
- [17] *React - A JavaScript library for building user interfaces*. <https://react.dev/>. Accessed: 2025-04-25. 2024.
- [18] *Tailwind CSS - Rapidly build modern websites without ever leaving your HTML*. <https://tailwindcss.com/>. Accessed: 2025-04-25. 2024.
- [19] *Redis - In-Memory Data Structure Store*. <https://redis.io/>. Accessed: 2025-04-25. 2024.
- [20] *Celery - Distributed Task Queue*. <https://docs.celeryq.dev/en/stable/>. Accessed: 2025-04-25. 2024.
- [21] *Mistral AI - Open-Weight Language Models*. <https://mistral.ai/>. Accessed: 2025-04-25. 2024.
- [22] *Tavily Search API - Real-time Web Data for LLMs*. <https://docs.tavily.com/>. Accessed: 2025-04-25. 2024.
- [23] *Google Cloud Console - Manage Cloud Resources*. <https://console.cloud.google.com/>. Accessed: 2025-04-25. 2024.